Modern Cryptography and AI/ML Security

LECTURE 27

Garbled Circuits

IISER THIRUVANANTHAPURAM

Course ID: DSC 4110/6111 **Instructor:** Dr. Laltu Sardar **Date:** October 13, 2026

1 Introduction

We have studied Beaver-triple-based multiplication of secret shares and developed a privacy-preserving inference model for computing $\mathbf{w}^T\mathbf{x} + b$ using two non-colluding, semi-honest servers. However, if one or more servers may behave maliciously, stronger security mechanisms are required. Threshold secret-sharing-based multiplication techniques, such as those used in multiparty computation (MPC), can provide security against a bounded number of malicious parties, but generally require multiple servers and an appropriate threshold configuration.

This raises a natural question: *Can we perform privacy-preserving computation using only one computational server, without requiring multiple servers to hold shares of the private inputs?* Garbled circuits provide an alternative approach. They allow a party to evaluate a function on private inputs by receiving a cryptographically encoded Boolean circuit and suitable input wire labels, with oblivious transfer enabling the evaluator to obtain labels for its own input. Thus, garbled circuits support secure two-party computation without requiring a distributed set of computation servers, although their security guarantees depend on the underlying cryptographic primitives and the assumed adversarial model.

2 Functionality as a Boolean Circuit.

A functionality can be represented as a Boolean circuit if it can be computed using a finite composition of Boolean gates, such as AND, OR, and NOT. Formally, let $f : \{0,1\}^n \to \{0,1\}^m$ be a function. A Boolean circuit C_f computes f if $C_f(x) = f(x)$ for every $x \in \{0,1\}^n$. For example, equality testing, integer comparison, and arithmetic operations on binary inputs can be implemented using Boolean circuits.

A function is **computable** if an algorithm exists that produces the correct output and terminates for every valid input. However, not every mathematically defined function is computable. For example, the *halting problem*, which asks whether a given program terminates on a given input, is undecidable: no algorithm can answer this question correctly for all programs and inputs. Consequently, no Boolean circuit family can compute the halting function for arbitrary input lengths. A fixed finite circuit computes a function on inputs of a fixed length, whereas computations over arbitrarily long inputs are represented by families of circuits $\{C_n\}_{n\geq 1}$.

For secure computation, *efficient circuit representation* is particularly important. Every polynomial-time computable function admits a polynomial-size circuit family, satisfying $C_n(x) = f_n(x)$ and $|C_n| = \text{poly}(n)$, where $|C_n|$ denotes the number of gates. Although some computable functions may require superpolynomial circuit size, making their evaluation expensive, polynomial-size circuits provide a natural setting for practical garbled-circuit protocols. Thus, the feasibility of garbled circuits depends on whether the desired functionality admits an efficient Boolean circuit representation.

Example: Converting a Function into a Boolean Circuit. Consider the function $f : \{0,1\}^2 \to \{0,1\}$ defined by $f(x_1, x_2) = x_1 \wedge x_2$. This function can be represented by a single AND gate, whose inputs are x_1, x_2 and whose output is $z = x_1 \wedge x_2$. As a more practical example, consider equality testing between two n-bit strings $x = (x_1, \ldots, x_n)$ and $y = (y_1, \ldots, y_n)$. The equality function $f_{\mathsf{eq}} : \{0,1\}^{2n} \to \{0,1\}$ is defined as

$$f_{\mathsf{eq}}(x, y) = \begin{cases} 1, & \text{if } x = y, \\ 0, & \text{otherwise.} \end{cases}$$

For each $i \in \{1, \ldots, n\}$, compute $e_i = \neg(x_i \oplus y_i)$, where $\oplus$ denotes XOR. The output is then

$$z = \bigwedge_{i=1}^{n} e_i = \bigwedge_{i=1}^{n} \neg(x_i \oplus y_i).$$

Thus, the equality function can be implemented using n XOR gates, n NOT gates, and $n-1$ AND gates. The resulting circuit has size $O(n)$. In a garbled-circuit protocol, Alice and Bob can evaluate this circuit on their private strings without directly revealing the strings to each other, subject to the security guarantees of the protocol.

3 Garbled Circuits

Garbled circuits (GC) are a fundamental *technique* for secure two-party computation. The central idea is to allow one party to encode a computation cryptographically, so that the other party can evaluate the function without learning the intermediate values.

3.1 System Model.

Consider two parties, Alice and Bob, who wish to compute a publicly known function $f : \{0,1\}^n \times \{0,1\}^m \to \{0,1\}^\ell$. Alice holds a private input $x \in \{0,1\}^n$, while Bob holds a private input $y \in \{0,1\}^m$. Their objective is to compute $z = f(x,y)$ while preserving the privacy of their respective inputs.

Let C be a Boolean circuit implementing f. Alice acts as the *garbler*, generating a garbled circuit $\widetilde{C}$ and the corresponding wire labels. Bob acts as the *evaluator*, using the garbled circuit and the labels corresponding to the parties' inputs to evaluate C without directly accessing the underlying intermediate Boolean values.

3.2 Communication Model.

Alice sends the garbled circuit $\widetilde{C}$ and the wire labels corresponding to her input x to Bob. For Bob's input y, the parties execute a 1-out-of-2 oblivious transfer (OT) protocol for each input bit. This enables Bob to obtain the labels corresponding to his input bits without revealing his choices to Alice. Bob then evaluates $\widetilde{C}$ and decodes the output $z = f(x,y)$.

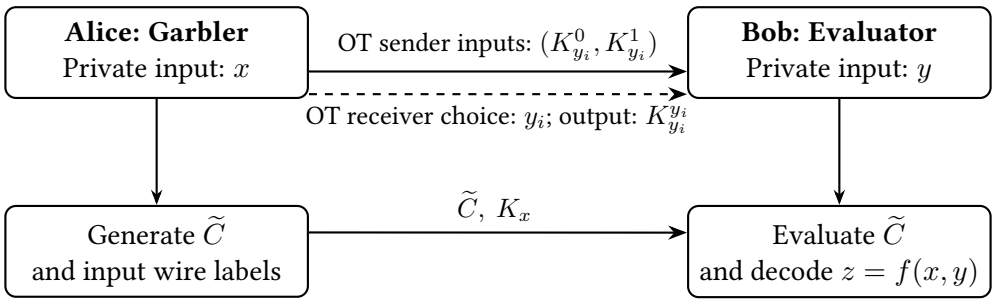


Figure 1: Communication model for garbled-circuit evaluation.

3.3 Adversarial Model.

We consider the semi-honest adversarial model, in which both parties follow the prescribed protocol but may attempt to infer additional information from their respective views. Alice may try to learn Bob's input from the protocol transcript, while Bob may try to recover Alice's input or the intermediate values of the computation. The protocol relies on the security of the garbling scheme and the OT protocol to protect the parties' private inputs.

3.4 Security Goals.

The protocol aims to achieve the following properties:

- **Correctness:** Bob correctly obtains $z = f(x,y)$.

- **Input privacy:** Alice learns no additional information about y, and Bob learns no additional information about x, beyond what is implied by their respective inputs and authorized outputs.
- **Intermediate-value privacy:** Bob evaluates the circuit using cryptographic wire labels without directly learning the intermediate Boolean values.

These guarantees depend on the security of the underlying cryptographic primitives. The basic protocol provides security against semi-honest adversaries; protection against malicious parties requires additional mechanisms to enforce correct protocol execution.

3.5 Protocol description

1. Boolean Circuit and Truth Table. Let C be a Boolean circuit computing $f : \{0,1\}^n \times \{0,1\}^m \to \{0,1\}^\ell$. For illustration, consider the AND function $f(x, y) = x \wedge y$.

Its truth table is given below.

x	y	$z = x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

A conventional evaluation requires the evaluator to know the input bits. In a garbled circuit, however, Bob receives cryptographic labels representing the bits and evaluates the gate without directly learning their values.

2. Wire Labels and Garbled Gates. Let W be the set of wires in C. For each wire $w \in W$, Alice independently generates two random labels

$$K_w^0, K_w^1 \in \{0,1\}^\kappa,$$

where κ is the security parameter. The label K_w^0 represents bit 0, while K_w^1 represents bit 1.

The labels are not the bits themselves. They are random-looking cryptographic strings that identify the values needed for evaluating the circuit. Knowing K_w^0 does not automatically reveal K_w^1.

For the AND gate, let u and v be the input wires and w be the output wire. Alice generates the four labels

$$K_u^0, K_u^1, K_v^0, K_v^1$$

for the input wires, and K_w^0, K_w^1 for the output wire.

The gate computes $w = u \wedge v$. Alice constructs an encrypted entry for every possible pair of input bits (a, b):

$$T_g[a, b] = \mathsf{Enc}_{K_u^a, K_v^b}\left(K_w^{a \wedge b}\right).$$

Here, $\mathsf{Enc}_{K_u^a, K_v^b}$ denotes a secure garbling encryption operation using both input labels as key material. In a practical implementation, a standard garbling construction such as point-and-permute specifies the exact encryption and decryption operations.

The corresponding table is conceptually represented as follows.

Input u	Input v	Output bit	Garbled entry
0	0	0	$\mathsf{Enc}_{K_u^0, K_v^0}(K_w^0)$
0	1	0	$\mathsf{Enc}_{K_u^0, K_v^1}(K_w^0)$
1	0	0	$\mathsf{Enc}_{K_u^1, K_v^0}(K_w^0)$
1	1	1	$\mathsf{Enc}_{K_u^1, K_v^1}(K_w^1)$

The table describes the four encrypted entries. Alice shuffles them or uses a suitable point-and-permute mechanism so that Bob cannot identify the logical input pair merely from the table position.

3. Protocol Overview. The protocol consists of four steps.

1. **Garbled circuit generation:** Alice generates the wire labels and garbles every gate of C. She sends the garbled circuit $\widetilde{C}$ to Bob.
2. **Input-label delivery:** For Alice's input $x = (x_1, \ldots, x_n)$, she sends Bob the active labels
$$K_x = (K_{x_1}^{x_1}, \ldots, K_{x_n}^{x_n}).$$
For Bob's input $y = (y_1, \ldots, y_m)$, the parties execute 1-out-of-2 oblivious transfer for every input bit y_i. Alice acts as the sender with messages $(K_{y_i}^0, K_{y_i}^1)$, and Bob acts as the receiver with choice bit y_i. Bob obtains only $K_{y_i}^{y_i}$.
3. **Circuit evaluation:** Bob evaluates $\widetilde{C}$ gate by gate using the active input labels. Each gate produces the active label for its output wire. Bob does not need to recover the intermediate Boolean bits.
4. **Output decoding:** Alice provides suitable output decoding information. Bob uses it to decode the active output label and obtain $z = f(x, y)$.

3.6 Worked Example: Evaluating an AND Gate.

Suppose Alice holds $x = 1$ and Bob holds $y = 0$. The correct output is
$$z = x \wedge y = 1 \wedge 0 = 0.$$

For simplicity, suppose the active labels are
$$K_u^1 \quad \text{and} \quad K_v^0.$$

Alice's garbled table contains the entry
$$T_g[1, 0] = \mathsf{Enc}_{K_u^1, K_v^0}(K_w^0).$$

Bob possesses the two active input labels. Using them, he identifies and decrypts the corresponding valid garbled entry to obtain K_w^0. He does not need to know that the underlying input pair was $(1, 0)$.

Finally, the output decoding information maps K_w^0 to the bit 0. Thus Bob obtains the correct output.

The essential point is that Bob evaluates the gate using labels, not by recovering the actual input bits. In a secure garbling scheme, the labels and encrypted tables conceal the intermediate computation, subject to the security guarantees of the construction.

3.7 Correctness.

For every gate g with input wires u, v and output wire w, the garbled gate is constructed so that valid active input labels produce the correct active output label:
$$\mathsf{Eval}\big(\widetilde{g}, K_u^a, K_v^b\big) = K_w^{g(a,b)}.$$

Applying this property inductively in topological order over the circuit shows that Bob obtains the correct active label on every wire. Consequently, output decoding yields
$$\mathsf{Decode}(K_{\text{out}}) = f(x, y).$$

3.8 Security Intuition.

In the semi-honest model, both parties follow the protocol but may attempt to infer additional information from their views.

- **Privacy of Alice's input:** Bob receives only the active labels for Alice's input wires. The garbled circuit conceals the intermediate values and inactive labels.
- **Privacy of Bob's input:** Oblivious transfer ensures that Alice does not learn Bob's OT choice bits, while Bob obtains only the selected input labels.

- **Output leakage:** The output itself may reveal information about the inputs. Garbled circuits do not eliminate information logically implied by the authorized output.

These guarantees rely on the security of the garbling scheme and OT. The basic protocol described here targets semi-honest adversaries. Malicious security requires additional mechanisms to ensure that the parties follow the prescribed computation.

References

[1] B. Darvish Rouhani, M. Sadegh Riazi, and F. Koushanfar, "DeepSecure: Scalable Provably-Secure Deep Learning," *arXiv preprint arXiv:1705.08963*, 2017.